Singly Linked List – Auxiliary Search and Time Complexity.

Program:

```
void auxiliarySearch(Node **head, int key, Node **ploc, Node **loc)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if ((*head)->data == key)
    {
        *loc = *head;
        return;
    }

    *ploc = *head;
    *loc = (*head)->next;

    while (*loc != nullptr)
    {
        if ((*loc)->data == key)
        {
            return;
        }
        else
        {
            *ploc = *loc;
            *loc = (*loc)->next;
        }
    }
```

```
}

…..
        case 12:
        {
            Node *ploc = nullptr, *loc = nullptr;
            int key;
            cout << "Enter the element to search: ";
            cin >> key;
            auxiliarySearch(&head, key, &ploc, &loc);
            if (loc != nullptr)
            {
                cout << "Element " << key << " found." << endl;
                if (ploc != nullptr && loc != nullptr)
                {
                    cout << "Previous element is " << ploc->data << "." << endl;
                    cout << "Searched Element is " << loc->data << "." << endl;
                }
                else
                {
                    cout << loc->data << " is at the head of the list." << endl;

                    cout << "Previous Element is absent ,and ploc's value is
                    nullptr." << endl;
                }
            }
            else
            {
                cout << "Element " << key << " not found." << endl;
            }
            break;
        }
```

Starting from the first from Switch – Case:

```
        case 12:
        {
            Node *ploc = nullptr, *loc = nullptr;
    …….
        }
```

$Node * ploc$	
$Address$	$ploc$
$ff31$	$nullptr$

$Node * loc$	
$Address$	loc
$ff32$	$nullptr$

$Now\ , lets\ say\ Key = 19, which\ is\ an\ user\ input.$

```
        case 12:
        {
            Node *ploc = nullptr, *loc = nullptr;
            int key;
            cout << "Enter the element to search: ";
            cin >> key;
            auxiliarySearch(&head, key, &ploc, &loc);
           ……
        }
```

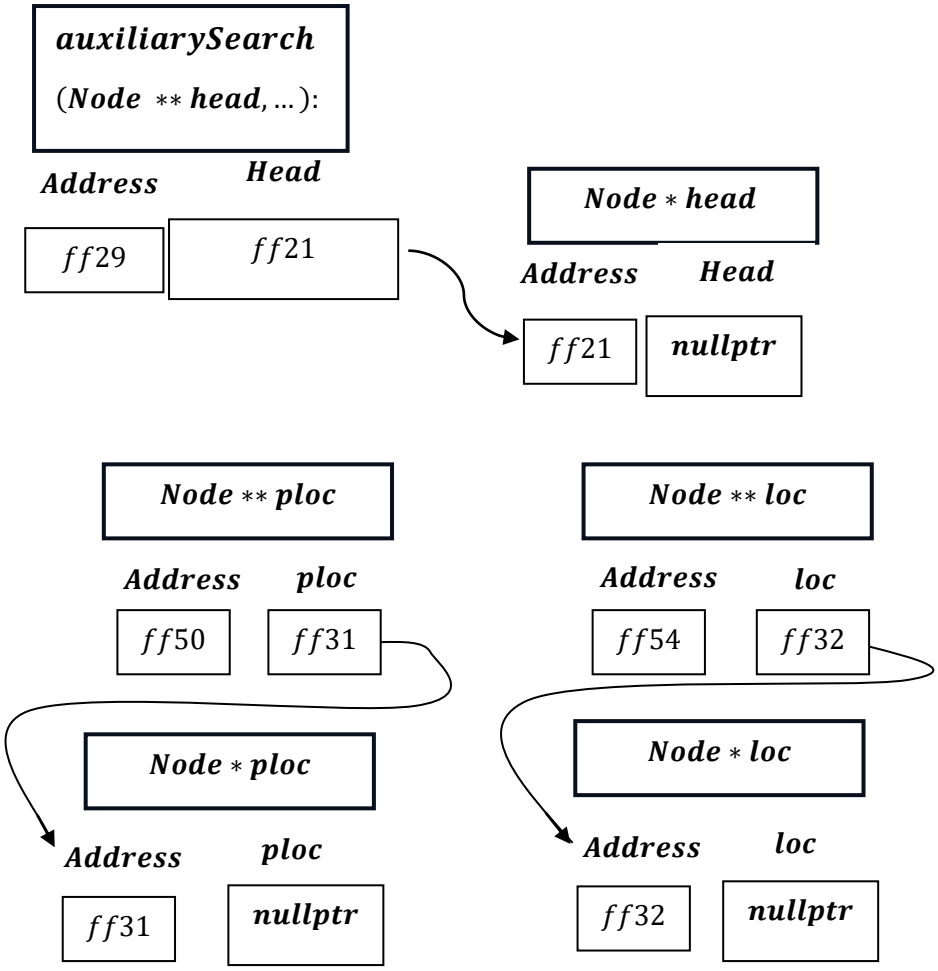
$Now, say\ our\ list\ is\ empty$:

$Node * head$	
$Address$	$Head$
$ff21$	$nullptr$

$auxiliarySearch(ff21, 19, ff31, ff32);$

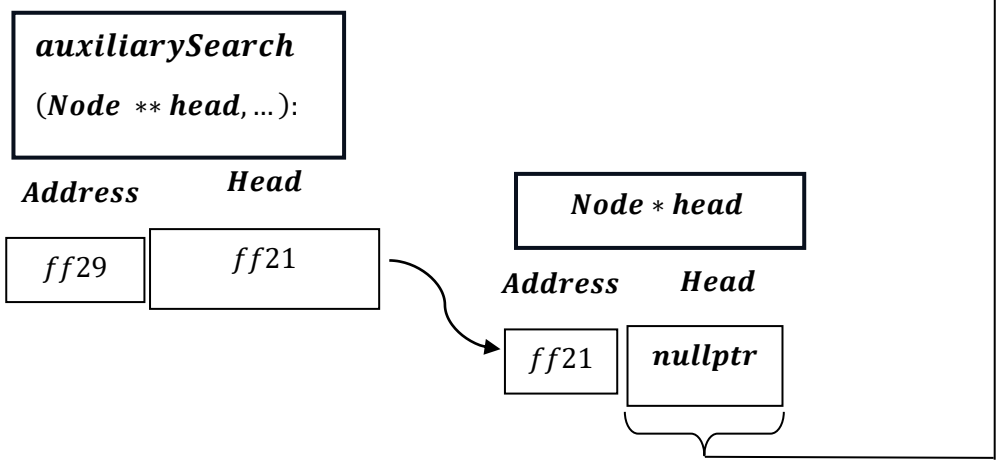
Now,

```
void auxiliarySearch(Node **head, int key, Node **ploc, Node **loc) {…}
```



$if(*head == nullptr)$ *if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the auxiliarySearch function.

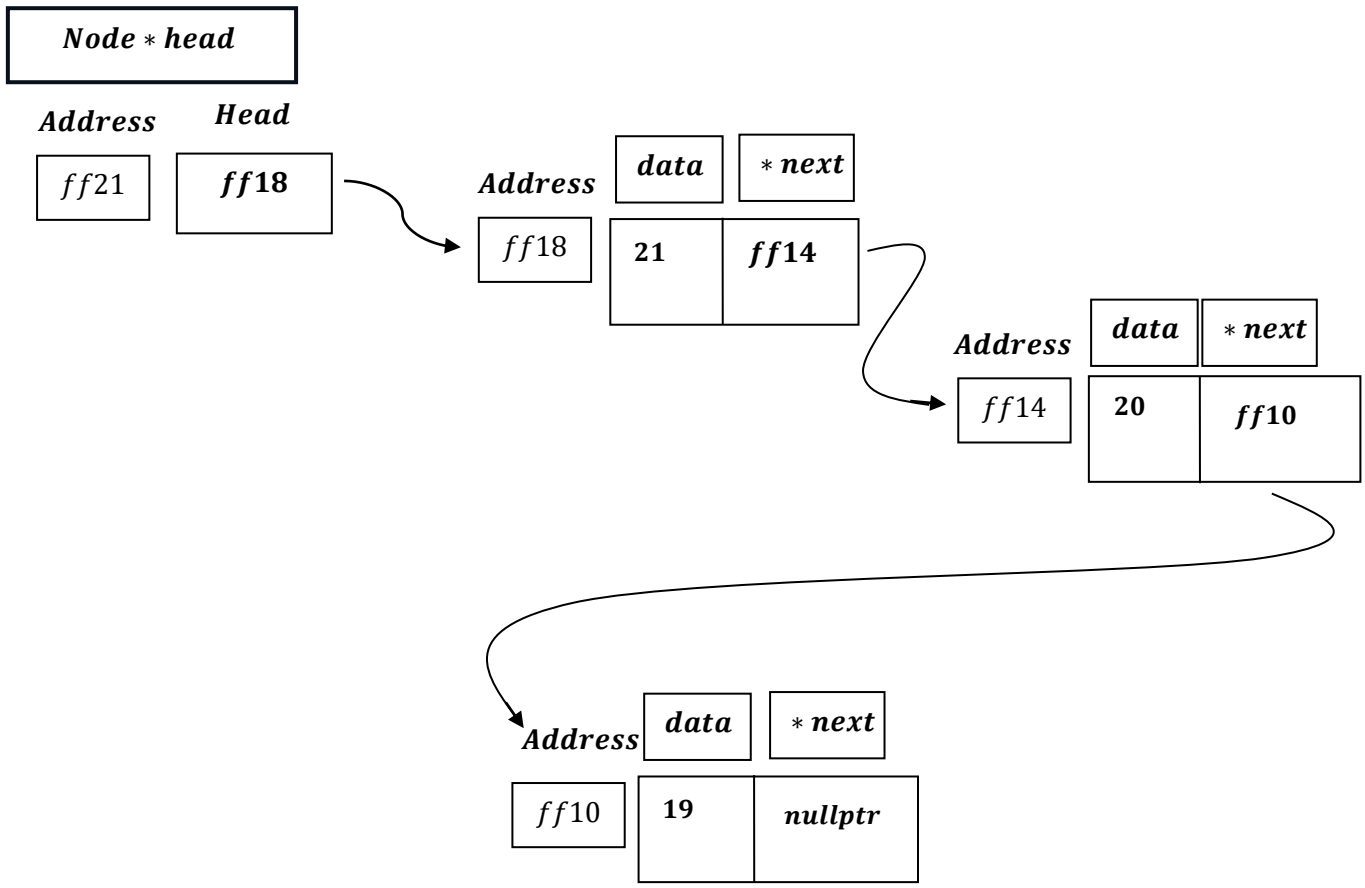



```
if (*head == nullptr)
  {
      cout << "Error: List is empty." << endl;
      return;
  }
```

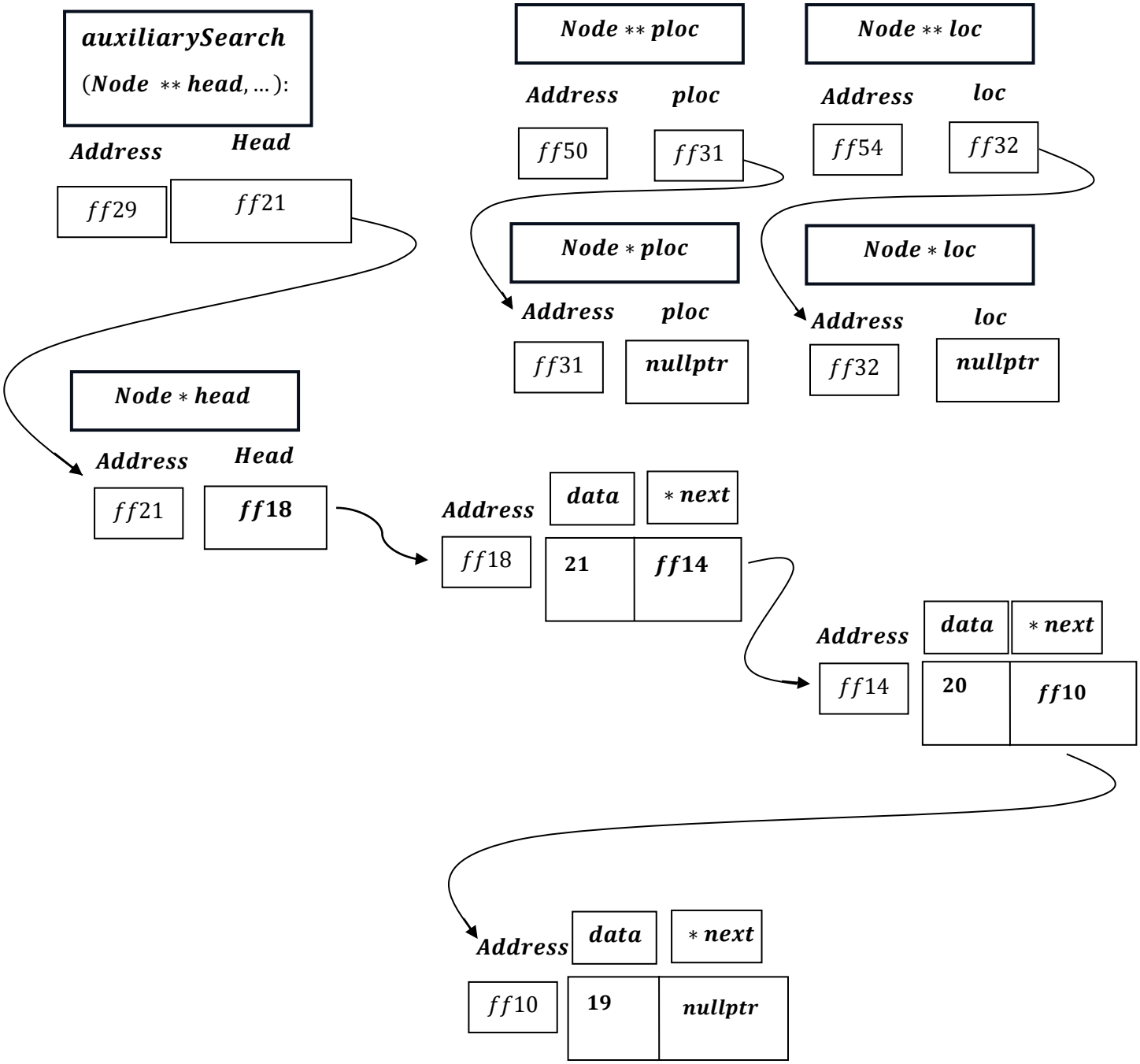
If List is Empty , then , it prints a message : `Error: List is empty.` Here input position entered by user doesn't matters.

Now if Linked List is not Empty:

We have linked list :



Now,



$Now, if\ ((*head) \rightarrow data == key)\{\ldots\} i.e.\ first\ key\ is\ available\ in\ first\ node$

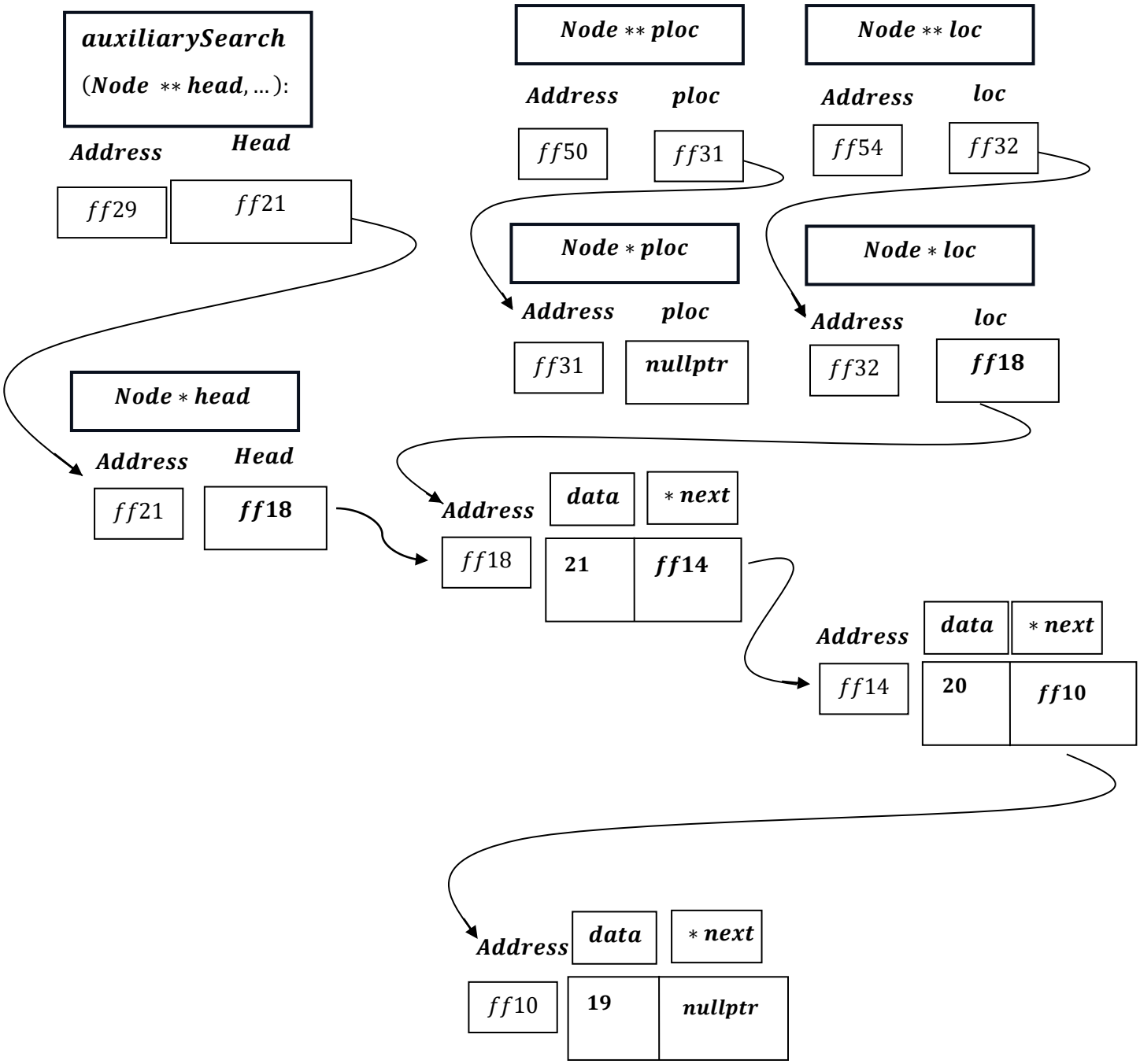
$then$:

$if\ ((*head) \rightarrow data == key)\{$

$*loc = *head;$

return;

}



function call: ***auxiliarySearch***(&***head*** : ***ff21***, ***Key***: **21**, &***ploc***: ***ff31***, &***loc***: ***ff32***);

Next:

```
            if (loc != nullptr)
            {
                cout << "Element " << key << " found." << endl;
                if (ploc != nullptr && loc != nullptr){…}
                else
                {
                    cout << loc->data << " is at the head of the list." << endl;

                    cout << "Previous Element is absent ,and ploc's value is
                    nullptr." << endl;
                }
            }
```

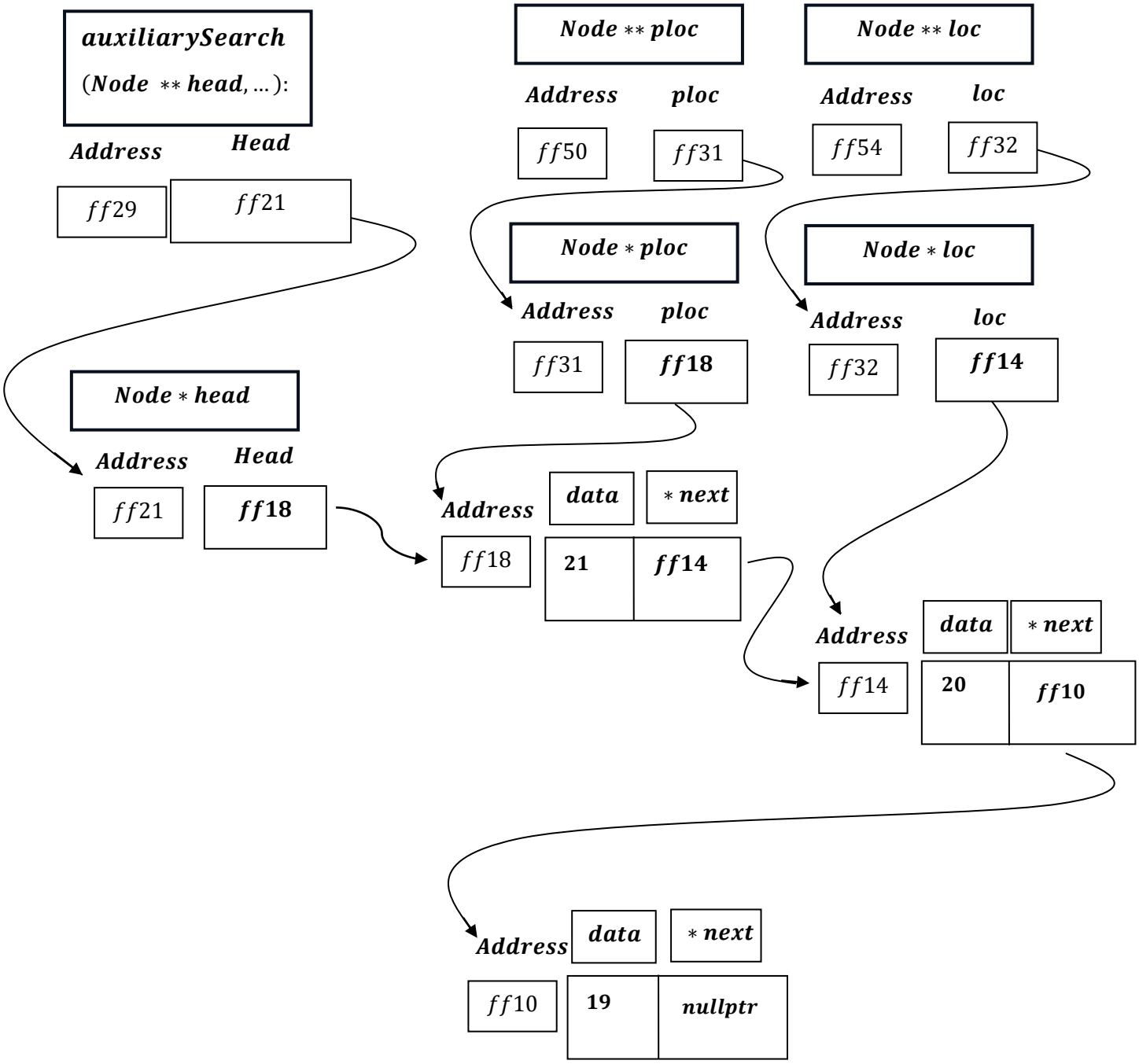
As `loc` is not nullptr , hence , it prints " Element 21 is found."

ploc pointer is at nullptr while loc pointer is not . Hence if condition becomes false and it goes to else part:

After moving to else part , it prints "21 is at head of the list."
"Previous Element is absent , and ploc's value is nullptr."

Now, if Searched Key locate beyond Head Node

```
    *ploc = *head;
    *loc = (*head)->next;
```



$Now, if\ the\ function\ call\ is$: $auxiliarySearch(ff21, 19, ff31, ff32)$;

$Next,$

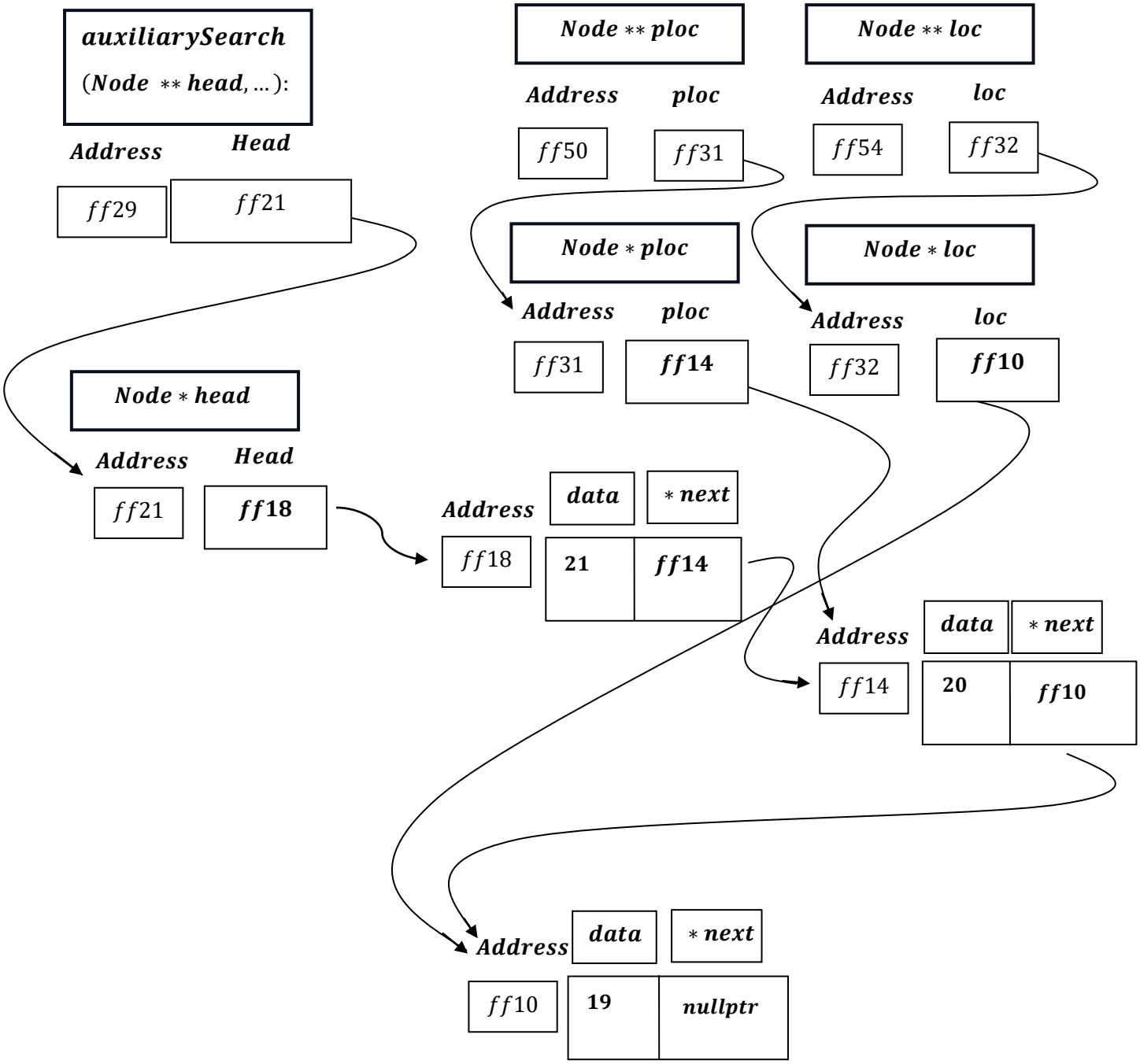
```
while (*loc != nullptr)
{
    if ((*loc)->data == key)
    {
        return;
    }
    else
    {
        *ploc = *loc;
        *loc = (*loc)->next;
    }
}
```

$As\ pointer\ loc\ doesnot\ contain : Nullptr, hence\ \ it\ checks\ first:$

$if\big((*loc) \rightarrow data == key\big)\{return;\}\ which\ is\ not\ true\ at\ current\ case,$

$hence\ it\ moves\ to\ else\ part\ and\ assigns : *ploc = *loc = ff14\ and\ *loc = (*loc) \rightarrow next = ff10;$

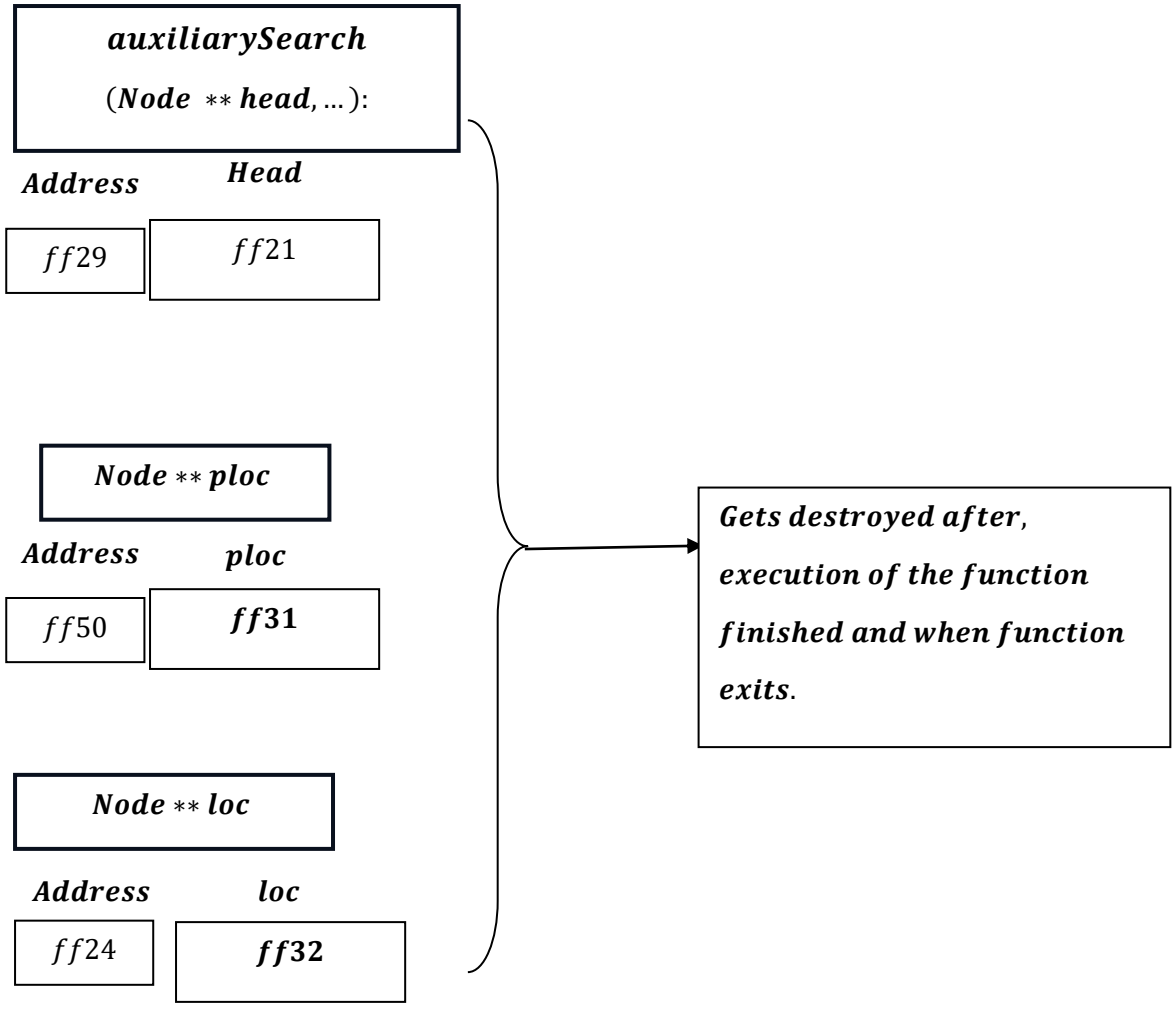
$Which\ we\ can\ depict\ through\ pictorally,$



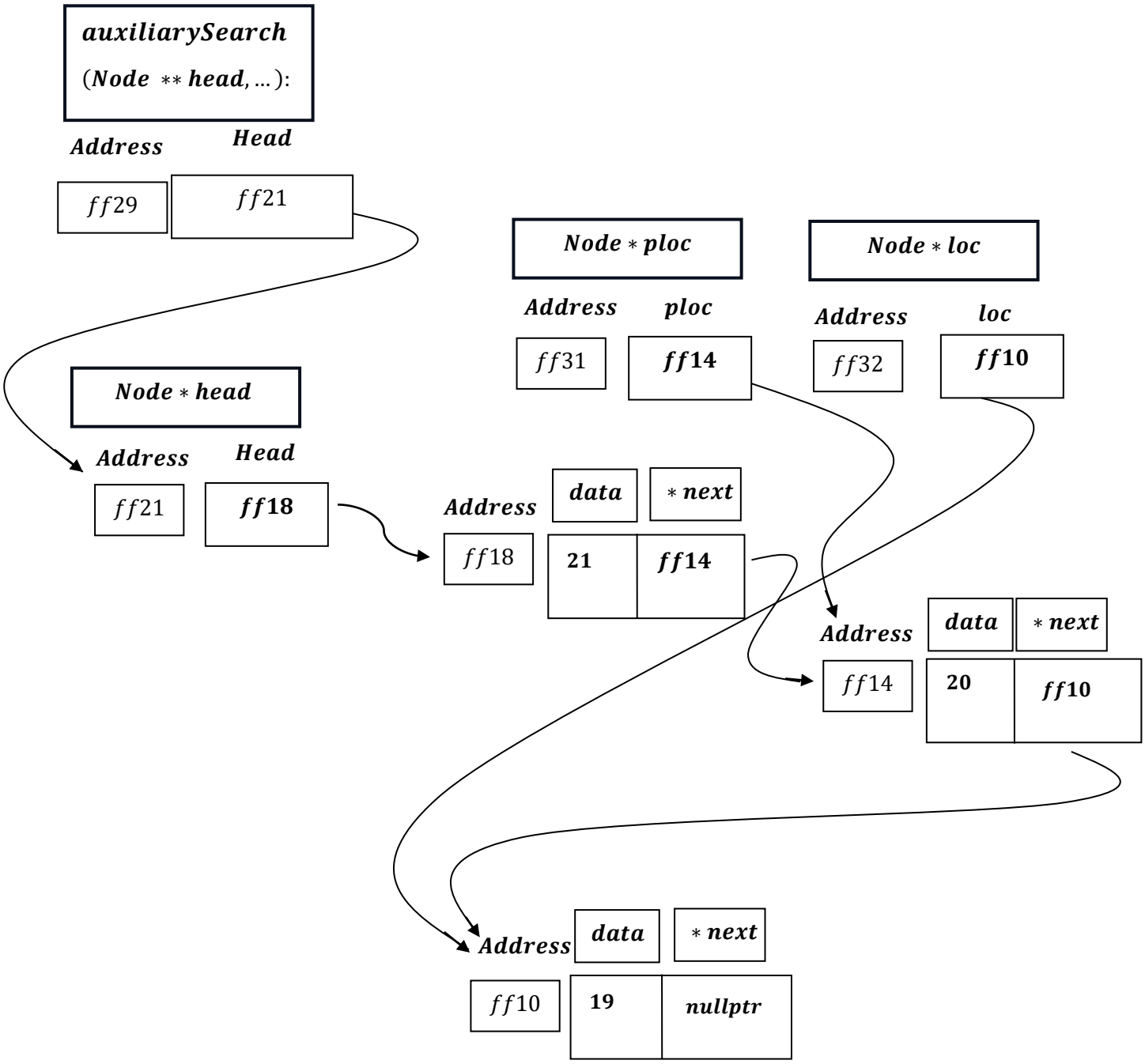
$$\textbf{Now, if}\big((*\,loc) \rightarrow data == key\big)\{return;\}\ \textbf{which is true at}$$

$current\ case, as\ (*loc) \rightarrow data = 19 = Key$. $Hence\ the\ function\ stops\ here.$

$After\ the\ function\ module\ ends$, $all\ temporary\ pointer\ variables\ created\ gets\ destroyed, i.e.$



$And\ we\ are\ left\ with$:


auxiliarySearch
(Node ** head, ...):
Address
Head
ff29
ff21
Node * ploc
Address
ploc
ff31
ff14
Node * loc
Address
loc
ff32
ff10
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr

Now,

```
        if (loc != nullptr)
        {
            cout << "Element " << key << " found." << endl;
            if (ploc != nullptr && loc != nullptr)
            {
                cout << "Previous element is " << ploc->data << "." << endl;
                cout << "Searched Element is " << loc->data << "." << endl;
            }
            else
            {….}
        }
```

Now as , loc pointer is not equals to nullptr, which makes if condition to true, then, it prints "Element 19 is found."

Now, as , loc pointer and ploc pointer is both not null , that indicates , we data found is beyond Head Node and we have previous and current data , i.e. loc → data = 19 and ploc → data = 20, then:

It prints: "Previous element is 20 ."
"Searched element is 19 ."

Now when Key is searched but not found

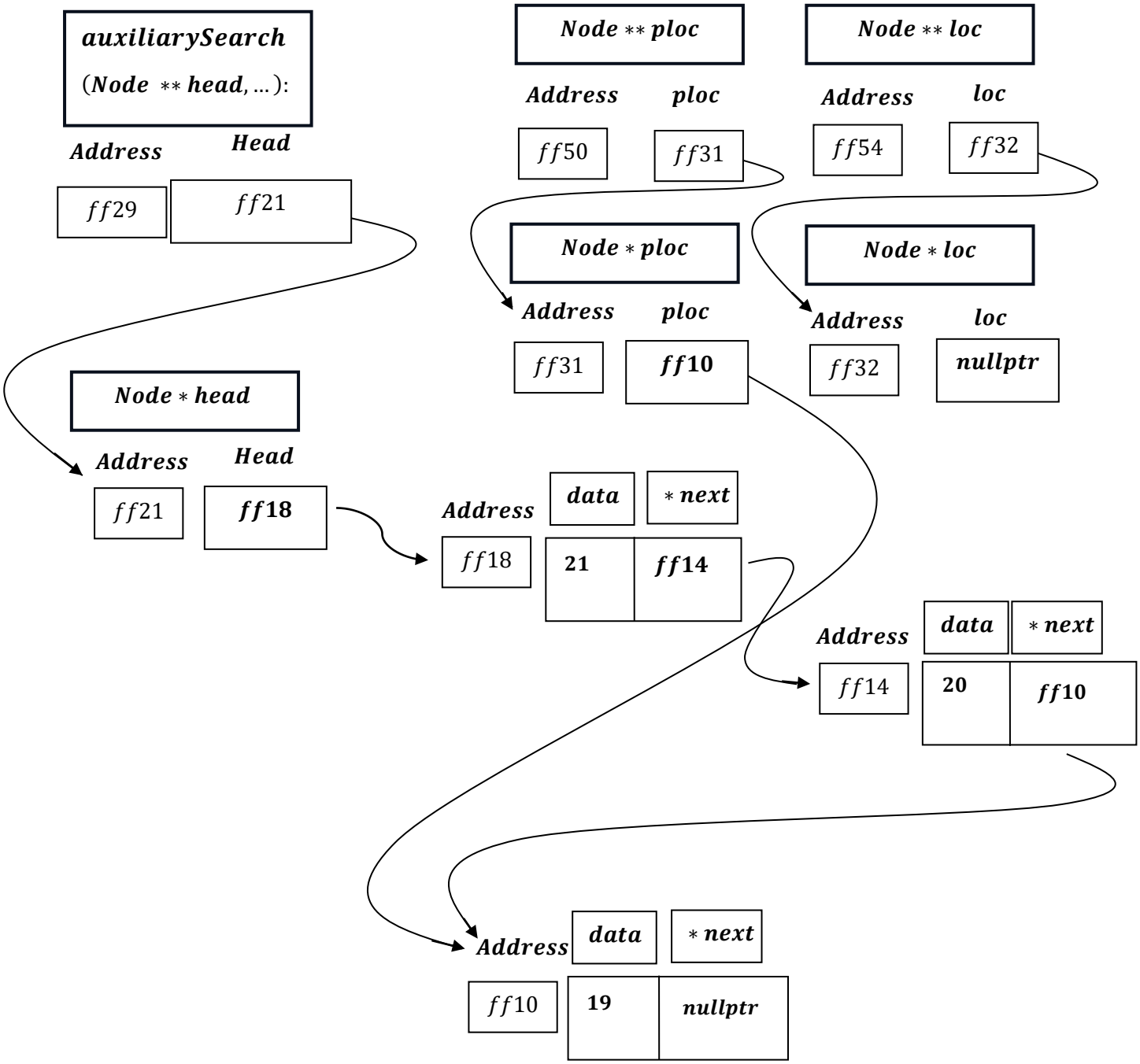
function call: auxiliarySearch(&head : ff21, Key: 18, &ploc: ff31, &loc: ff32);

Then, there is another loop execution and :

```
        *loc = (*loc)->next;
```

∗ loc = (∗ loc) → next = nullptr and ∗ ploc =∗ loc = ff10 i.e. ploc points to

last tail node , and loop execution ends as loop condtion loc not equals to nullptr becomes false.



Now as the function ends , and loc pointer becomes nullptr which indicates Key not found.

Hence as we finishes searching and comes back to switch – case of main function :

```
            if (loc != nullptr){….}
            else
            {
                cout << "Element " << key << " not found." << endl;
            }
```

if condition becomes false , hence it prints "Element 18 not found."

The function `auxiliarySearch` step by step explanation:

– The function first checks if the list is empty by checking if `* head` is `nullptr`.

– If the list is empty, it prints an error message and returns immediately. This prevents further operations on an empty list.

– The function then checks if the key is present in the head node by comparing `(* head) → data` with the key.

– If the key is found at the head node, it sets `* loc` to point to the head node and returns. This indicates that the node with the key has been found at the head.

– If the key is not at the head node, the function initializes `* ploc` to point to the head node.

– It also initializes `* loc` to point to the node following the head node

(`(* head) → next`).

– The function enters a `while` loop that continues as long as `*loc` is not `nullptr`.

– Inside the loop, it checks if the current node (`*loc`)contains the key by comparing `(*loc) → data` with the key.

– If the key is found, the function returns immediately, leaving `*ploc` and `*loc` pointing to the previous node and the current node, respectively.

– If the key is not found, the function updates `*ploc` to point to the current node (`*loc`), and then moves `*loc` to the next node (`(*loc) → next`).

Summary:

– Initialization: The function starts by checking if the list is empty and handles this case by printing an error message and returning.

– Head Node Check: It then checks if the key is at the head node. If so, it sets `*loc` to the head node and returns.

– Traversal: If the key is not at the head node, it initializes `*ploc` and `*loc` to point to the head node and the next node, respectively. It then traverses the list, updating `*ploc` and `*loc` as it goes, until it finds the key or reaches the end of the list.

Time Complexity of Auxiliary Search

```
void auxiliarySearch(Node **head, int key, Node **ploc, Node **loc)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if ((*head)->data == key)
    {
        *loc = *head;
        return;
    }

    *ploc = *head;
    *loc = (*head)->next;

    while (*loc != nullptr)
    {
        if ((*loc)->data == key)
        {
            return;
        }
        else
        {
            *ploc = *loc;
            *loc = (*loc)->next;
        }
    }
}
```

$O(1)$

$O(1)$

$O(1)$

$O(1)$

At worst case , loop runs $N-1$ times.

At worst case , check occurs $N-1$ times.

At worst case , when Key is found runs $N-2$ times , when Key is not found runs $N-1$ times.

Best Case

1)*When List is empty*:

$-$ ***The function first checks if the linked list is empty by examining the `* head` pointer.***

$-$ ***If `* head` is `nullptr`, it means the list is empty.***

$-$ ***An error message "Error: List is empty." is printed to the console.***

This whole process takes constant amount of time i.e. $\Omega(1)$.

2)*When Key present at Head node*:

$-$***if*** $(*\,head) \rightarrow$ ***data equals to `Key` , condition gets matched , then pointer variable*** $*$ ***loc gets assigned to pointer*** $*$ ***head.***

This whole process takes constant amount of time i.e. $\Omega(1)$.

Worst Case

1) *When `Key is Found` in Last Node*:

$-$***Now head pointer doesnot carries `Null Pointer ` value, hence it checks at if condition first , such checking takes constant amount of time i.e.*** $O(1)$.

$-When\ (*head) \rightarrow data\ is\ not\ the\ Key\ i.e.Key\ is\ not\ found\ at\ head\ node, this$
$check\ takes\ constant\ time\ complexity\ i.e.O(1).$

$-Now\ previous\ location\ pointer\ `ploc`\ points\ to\ Head\ Node, as\ `ploc`\ assigned\ to\ Head$
$Node\ address\ \ and\ current\ location\ pointer\ `loc`\ points\ to\ 2nd\ Node\ of\ the$
$Linked\ List\ as\ 2nd\ Node's\ address\ gets\ assigned\ to\ pointer\ `loc`\ , this$
$assignments\ takes\ \ constant\ time\ complexity\ i.e.O(1).$

$-Next, as\ key\ is\ found\ at\ last\ Node\ , Loop\ runs\ N-1\ times\ also:$

$-Whether\ \ (*loc) \rightarrow data\ equals\ to\ Key\ , this\ check\ occurs\ N-1\ times.$

$-*ploc = *loc\ and\ *loc = (*loc) \rightarrow next\ runs\ N-2\ times.$

$Hence\ , total\ complexity: N-1+N-2 = 2N-3\ .$

$And\ applying\ Big-O\ , we\ get: O(2N-3\ ) = O(N)\ .$

<u>1) *When `Key is Not Found` in Last Node*:</u>

$-Now\ head\ pointer\ doesnot\ carries\ `Null\ Pointer\ `\ value, hence\ it\ checks$
$at\ if\ condition\ first\ , such\ checking\ takes\ constant\ amount\ of\ time\ i.e.O(1).$

$-When\ (*head) \rightarrow data\ is\ not\ the\ Key\ i.e.Key\ is\ not\ found\ at\ head\ node, this$
$check\ takes\ constant\ time\ complexity\ i.e.O(1).$

$-Now\ previous\ location\ pointer\ `ploc`\ points\ to\ Head\ Node, as\ `ploc`\ assigned\ to\ Head$
$Node\ address\ \ and\ current\ location\ pointer\ `loc`\ points\ to\ 2nd\ Node\ of\ the$

$Linked\ List\ as\ 2nd\ Node's\ address\ gets\ assigned\ to\ pointer\ `loc`\ ,this\ assignments\ takes\ constant\ time\ complexity\ i.e.O(1).$

$-Next, as\ key\ is\ found\ at\ last\ Node\ ,Loop\ runs\ N-1\ times\ also:$

$-Whether\ (*loc) \rightarrow data\ equals\ to\ Key\ ,this\ check\ occurs\ N-1\ times.$

$-*ploc = *loc\ and\ *loc = (*loc) \rightarrow next\ runs\ N-1\ times.Till\ *loc\ assigned\ `NullPtr`\ .$

$Hence\ ,total\ complexity: N-1+N-1 = 2N-2\ .$

$And\ applying\ Big-O\ ,we\ get: O(2N-2\ ) = O(N)\ .$

Average Case

$(If\ the\ elements\ in\ the\ linked\ list\ are\ uniformly\ distributed\ and\ the\ key\ is\ equally\ likely\ to\ be\ any\ element\ in\ the\ list)$

$-if\ key\ is\ found\ at\ 1^{st}\ Node\ ,loop\ runs\ 0\ time\ ,Key\ searched\ for\ 1\ time\ .$

$-if\ key\ is\ found\ at\ 2^{nd}\ Node\ ,loop\ runs\ 1\ time\ ,Key\ searched\ for\ 2\ time\ .$

$-if\ key\ is\ found\ at\ 3^{rd}\ Node\ ,loop\ runs\ 2\ time\ ,Key\ searched\ for\ 3\ time\ .$

.

.

$-if\ key\ is\ found\ at\ N^{th}\ Node\ ,loop\ runs\ N-1\ time\ ,Key\ searched\ for\ N\ time\ .$

$As\ ,the\ key\ is\ equally\ likely\ to\ be\ any\ element\ in\ the\ list,$

$$probablity\ of\ finding\ 1^{st}\ element\ (p_1), if\ key\ equals\ to\ 1^{st}\ element\ = \frac{1}{N}.$$

$$probablity\ of\ finding\ 2^{nd}\ element\ (p_2), if\ key\ equals\ to\ 2^{nd}\ element\ = \frac{1}{N}.$$

.

.

$$probablity\ of\ finding\ N^{th}\ element\ (p_2), if\ key\ equals\ to\ N^{th}\ element\ = \frac{1}{N}.$$

$Therefore,$

$$Probability(P) = p_1 = p_2 = \cdots = p_n = \frac{1}{N}.$$

<u>1) *Approach based on loop execution and Key Searching*:</u>

$$loop\ execution\ occurs: \sum_{i=0}^{n-1} i\ \ or, a_0 + \sum_{i=1}^{n-1} i\ \ or\ 0 + \sum_{i=1}^{n-1} i\ \ (as, a_0 = 0\ )\ or\ \sum_{i=1}^{n-1} i$$

$$In\ loop\ execution\ , Key\ searching\ also\ takes\ place\ \sum_{i=1}^{n-1} i\ \ times.$$

$$Outside\ loop\ execution\ , Key\ Searching\ takes\ place\ only\ 1\ time\ .$$

$$Hence\ we\ can\ write\ \sum_{i=1}^{n-1} i + 1.$$

$$And\ Average\ case\ complexity =$$
$$(Number\ of\ times\ the\ loops\ executes\ with\ Key\ Searching$$
$$+ One\ extra\ outside) \times probablity\ of\ finding\ elements.$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow$ $\Theta(N)$ ***grows linearly with*** `$N$` .

$\rightarrow$ $\Theta\left(\frac{1}{N}\right)$ ***decreases to a constant , i.e. when N grows larger ,*** $\frac{1}{N}$ ***approaches to*** 0.

$\boldsymbol{This\ we\ can\ prove\ by\ limit\ and\ continuity\ also}$:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \ \left[\frac{c}{\infty} = 0\ , where\ c\ is\ constant\right].$$

$$As\ \Theta(N)'s\ growth\ rate\ is\ higher\ , therefore\ , average\ case\ is\ \Theta(N).$$

$$Hence, Average\ Time\ Complexity\ is:\ \Theta(N).$$

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get$:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

<u>2) $Approach\ based\ only\ based\ on\ Key\ Searching(alternative\ for\ above)$:</u>

$-if\ key\ is\ found\ at\ 1^{st}\ Node\ , Key\ searched\ for\ 1\ time$.

$-if\ key\ is\ found\ at\ 2^{nd}\ Node\ , Key\ searched\ for\ 2\ time$.

$-if\ key\ is\ found\ at\ 3^{rd}\ Node, Key\ searched\ for\ 3\ time$.

.

.

$-if\ key\ is\ found\ at\ N^{th}\ Node\ , Key\ searched\ for\ N\ time$.

$As\ , the\ key\ is\ equally\ likely\ to\ be\ any\ element\ in\ the\ list,$

$$probablity\ of\ finding\ 1^{st}\ element\ (p_1), if\ key\ equals\ to\ 1^{st}\ element\ = \frac{1}{N}.$$

$$probablity\ of\ finding\ 2^{nd}\ element\ (p_2), if\ key\ equals\ to\ 2^{nd}\ element\ = \frac{1}{N}.$$

.

.

$$probablity\ of\ finding\ N^{th}\ element\ (p_2), if\ key\ equals\ to\ N^{th}\ element\ = \frac{1}{N}.$$

$Therefore,$

$$Probability(P) = p_1 = p_2 = \cdots = p_n = \frac{1}{N}.$$

$Therefore, Average\ Case = Number\ of\ times\ the\ Key\ is\ searched\ \times$ $probablity\ of\ finding\ elements.$

$That\ is\ the\ average\ number\ of\ executions\ is\ the\ sum\ of\ all\ possible\ executions\ weighted\ by\ their\ probabilities. This\ is\ given\ by$:

$$\therefore Average\ Case = \frac{1}{N} \times \left(\sum_{i=1}^{N} i\right)$$

$$= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^{N} i = \frac{N(N+1)}{2}\right]$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

$\boldsymbol{Applying\ , Big\ Theta\ \Theta\ we\ get}$:

$$= \frac{1}{2} \times \left(\Theta(N) + \Theta(1)\right)$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

$\boldsymbol{Note\ , by\ relation}$: $\boldsymbol{\Omega \leq \Theta \leq O\ , we\ get}$:

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

Singly Linked List	***Cases***	***Time Complexity***
Auxiliary Search	**1.** ***Best Case***	$\Omega(1)$
	2. ***Worst Case***	$O(n)$
	3. ***Average Case***	$\Theta(n)$

****